# CLAVE: Custody-Less Accountable Verifiable Escrow

Rubén Sospedra

7 August 2026

**Abstract**

Platforms are increasingly required to identify their users, and they respond by storing identity documents they then hold indefinitely. The resulting database is a bulk surveillance capability wearing the costume of a targeted one. Refusing access instead wins the technical argument and loses the legislative one.

We first establish why the obvious middle path fails. No platform that holds sufficient key material can prove to a user that every use of that material was published. The argument rules out an entire family of designs, including any built on split keys, secure logs, or audited procedure, so long as one party holds every share.

CLAVE takes the only remaining direction that recruits no partner: the platform holds nothing. The user's device seals their legal identity against a public threshold network the platform does not operate. The platform stores ciphertext. A key for one account does not exist anywhere until a public on-chain request causes the network to derive it, and that key opens exactly one account. The network, not the platform, publishes a timelocked disclosure naming what was opened, so silence cannot be extended by inaction.

We give the construction, state the relation proved at enrollment, and analyse every link from the application binary through the circuit and the settlement chain to the escrow keys, separating what rests on cryptographic hardness from what rests on operational or economic assumptions. We contribute a verifiable escrow proof with 41.51 bits of soundness at the reference profile, eight design decisions each stated with the failure it prevents, and a cost model. Marginal cost is between `EUR` $1.6 \times 10^{-5}$ and $8.2 \times 10^{-4}$ per registration, four to five orders of magnitude below commercial identity verification.

A reference implementation accompanies this paper at `https://clave.sospedra.me`. It runs the protocol end to end against a mock settlement layer, and every figure below is emitted by one command. The zero-knowledge circuit is the exception and is evaluated in the clear, which we mark wherever it matters.

## 1 Introduction

Pseudonymity is being legislated away. Statutes now require identity checks before ordinary online activity. Platforms comply by collecting passports, and every breach corpus reads those collections for free.

The menu has two items. A surveillance database, or refusal. Refusal is correct and it keeps losing, because the demand underneath is legitimate. Fraud drains real accounts. Grooming happens in real conversations. Courts have real reasons to ask who was behind an account.

This work assumes access will exist, by law or by contract, and asks a narrower question. Given that the power exists, what must each use of it leave behind?

Certificate Transparency [2] made that move for certificates. Stop preventing misissuance, start proving it happened. CONIKS [3] carried it to key directories and named the honest guarantee as detection rather than prevention. Transparency overlays [4] generalise the treatment. All of them meet the same wall. A log proves what it contains. It cannot prove what it omits.

Abelson *et al.* [1] argued that mandated exceptional access carries risks outweighing its benefits. We accept the technical analysis. The political question did not settle, so the engineering question remains open.

We target three properties, stated so a non-specialist can hold a platform to them:

P1 The identity bound to an account is sealed.

P2 It can be unsealed only if the unsealing is published.

P3 The published record carries a reference a human can check by hand.

Section 2 shows that no platform holding the key material can offer P2, however carefully it is engineered. Sections 3 onward give a construction in which the platform holds no key material at all, and Section 4 analyses what secures each link of it.

This paper contributes:

1. an argument that custodial accountable pseudonymity is unachievable, and the three escape routes it leaves;
2. CLAVE, a construction that takes one of them, with the platform holding only ciphertext;
3. the enrollment relation stated formally, joining a credential proof to a verifiable escrow proof through one public commitment;
4. a link-by-link security analysis from application binary to escrow key, separating cryptographic guarantees from operational ones; and
5. a cost model showing marginal cost four to five orders of magnitude below commercial identity verification.

## 2 Why a custodian cannot do this

The natural design gives the platform a key, splits it in two, wraps every use in an append-only log, and anchors the log publicly. That design cannot deliver P2, and the reason is not an implementation flaw.

**Proposition 1** (Custodial impossibility)**.** *No system in which a single party holds sufficient key material can guarantee to a third party that every use of that material was published.*

*Argument.* Consider two worlds. In world A the platform follows the protocol and produces public transcript $T$. In world B the platform produces the identical transcript $T$, and also retains a copy of the plaintext, or a recovery key, or runs a second instance of its own software. Any verifier observes exactly $T$ in both worlds, so no function of $T$ separates them. □

Splitting the key across two modules the same party administers does not help, because the party holds both shares. Logging every use does not help, because the log records only what somebody chose to write. Anchoring the log does not help, because anchoring fixes history rather than compelling entries.

Cryptography constrains what can be computed. It does not constrain whether a computation happened in private. Publication is an act in the world, and forcing it requires the decryption key to depend on a value that (i) the platform cannot compute alone and (ii) comes into existence only when the record becomes public.

Exactly three sources satisfy both conditions. A threshold committee. Secure hardware with remote attestation, where the trust root becomes a silicon vendor. Witness encryption, which would satisfy the requirement exactly and does not exist in deployable form.

**Corollary.** The word "opening" needs care. Once plaintext reaches a platform's storage it can be reread indefinitely with no cryptography involved. The defensible event is the *first* record-specific key release, never every subsequent access. Any system claiming to log every access to an identity it holds in the clear is claiming something it cannot enforce.

# 3 Construction

## 3.1 Notation

Let $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ be groups of prime order $r$ with a bilinear pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$, and let $P_1, P_2$ generate $\mathbb{G}_1, \mathbb{G}_2$. Write $[x]P$ for scalar multiplication. Let $H_1 : \{0,1\}^* \to \mathbb{G}_1$ be a hash to the curve and $H : \{0,1\}^* \to \{0,1\}^{256}$ a collision-resistant hash.

The escrow network holds a master secret $\mathsf{msk} \in \mathbb{Z}_r$ shared among its nodes, and publishes $\mathsf{mpk} = [\mathsf{msk}]P_2$. For an identity string $\mathsf{id}$ the corresponding key is

$$\mathsf{sk}_{\mathsf{id}} = [\mathsf{msk}]\, H_1(\mathsf{id}), \tag{1}$$

which no party can compute without a threshold of shares. Encryption to $\mathsf{id}$ follows Boneh and Franklin [5]. Write $\mathsf{Enc}_{\mathsf{id}}(m; \rho)$ for encryption with coins $\rho$ and $\mathsf{Dec}_{\mathsf{sk}}(c)$ for decryption, which returns $\perp$ on failure.

## 3.2 Parties and assumptions

The adversary is the platform. It runs enrollment, storage, and its own transparency record. It is rational and wants one account's identity without a matching public entry.

A1 A threshold of the escrow network's nodes does not collude.

A2 The settlement chain does not reorganise below the finality depth set in policy.

A3 Bilinear Diffie-Hellman is hard in the chosen groups, discrete log is hard in $\mathbb{G}_1$, and $H$ is collision resistant.

A4 At least one party reads the published record against the chain, at least once. Continuous monitoring is not assumed.

A5 The passport chip's authentication key is not extractable.

A6 The user's device runs the software the platform published.

A1, A2, A5 and A6 are not cryptographic. Section 4.6 separates them out explicitly, because a reader deserves to know which parts of the construction rest on mathematics and which rest on people.

## 3.3 Overview

Figure 1 shows the four systems. The platform appears once and holds no secret.

## 3.4 Enrollment

Enrollment runs entirely in a native application, because passport chips require command exchanges that browser interfaces cannot issue. Figure 2 gives the order, and every arrow leaving the device carries ciphertext or a proof.

The application reads the chip and requires Active Authentication or Chip Authentication, so that a copied data dump does not suffice. The chip challenge $\mathsf{ch}(a, \eta)$ derives from the account identifier $a$ and a single-use server nonce $\eta$, and both are public inputs to the proof.

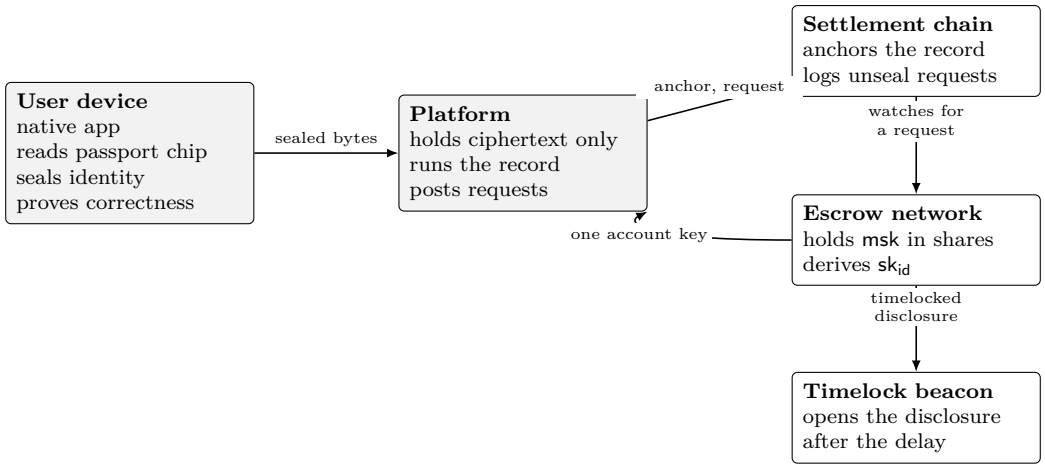


Figure 1: The four systems. Only the chain is written to by the platform. The escrow network is a set of machines that watches it. The beacon is a signing service, not a chain. Deployable instances of all three exist today.

The application samples a uniform $s \in \mathbb{Z}_r$, seals the identity under a key derived from $s$, shares $s$, encrypts each share to the escrow network, and produces two proofs joined by the public value $hS = [s]P_1$.

### 3.5 The enrollment relation

**Definition 1** (Enrollment relation $\mathcal{R}$)**.** *Public inputs are the account identifier $a$, the server nonce $\eta$, the commitment $hS \in \mathbb{G}_1$, the nullifier* $\mathsf{nf}$*, the sealed identity* $\mathsf{ct}$*, and the set $\mathcal{C}$ of trusted country signing keys. The witness is the passport data $D$, the country signature $\sigma_C$, the chip signature $\sigma_{AA}$, the scalar $s$, and the sealing nonce $\nu$. $\mathcal{R}$ holds when all of the following hold.*

$$\exists\, \mathsf{pk}_C \in \mathcal{C} : \mathsf{Ver}(\mathsf{pk}_C,\, D,\, \sigma_C) = 1 \tag{2}$$

$$\mathsf{pk}_{AA} = \mathsf{extract}_{AA}(D) \tag{3}$$

$$\mathsf{Ver}(\mathsf{pk}_{AA},\, \mathsf{ch}(a, \eta),\, \sigma_{AA}) = 1 \tag{4}$$

$$\mathsf{nf} = H(\mathsf{doc}(D) \,\|\, \mathsf{ctx}) \tag{5}$$

$$hS = [s]P_1 \tag{6}$$

$$\mathsf{ct} = \mathsf{AEAD.Enc}\big(\mathsf{kdf}(s),\, \nu,\, \mathsf{idb}(D),\, a\big) \tag{7}$$

Each line does one job. Equation 2 establishes that a state issued the document. Equation 3 takes the chip's authentication key from inside the signed data, so that key inherits the country's signature. Equation 4 establishes that the physical chip answered a challenge bound to this account and this session, which a copied data dump cannot do. Equation 5 derives the nullifier from the document alone, never from $a$, so one document yields one account. Equations 6 and 7 are the join: the identity signed by the state is exactly what is sealed, under exactly the key committed by $hS$.

The last two carry the weight. Without 7 a prover could present a genuine passport and seal unrelated bytes, and both proofs would verify independently. Because $D$ feeds 2 and 7 through the same witness wires, no such separation exists.

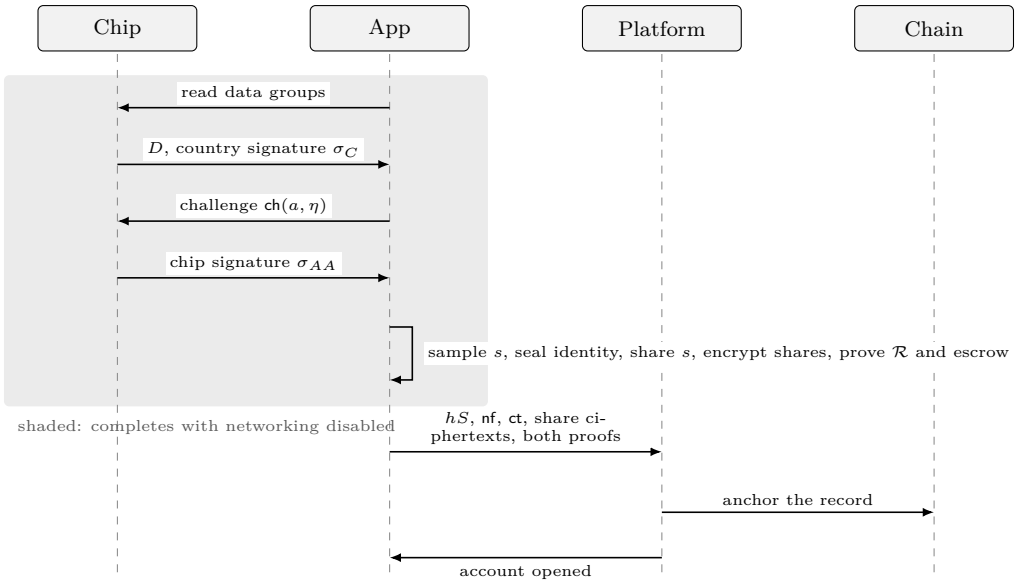


Figure 2: Enrollment. The identity never leaves the device. Sealing and proving complete with no network, so a user may perform the whole shaded region in airplane mode and reconnect only to transmit the result.

### 3.6 The verifiable escrow proof

$\mathcal{R}$ says the identity is sealed under $s$. It says nothing about whether $s$ can ever be recovered. That is the escrow proof's job.

Proving pairing-based encryption correct inside a general proof system is expensive, so we use cut-and-choose. The prover shares $s$ into $n$ shares at threshold $k$ [6] using a polynomial $f$ of degree $k-1$ with $f(0) = s$, publishes Feldman commitments $C_j = [c_j]P_1$ to the coefficients of $f$ [7], and encrypts each share to the account's identity string:

$$E_i = \mathsf{Enc}_{\mathsf{id}_a}(f(i);\, \rho_i), \qquad i = 1, \ldots, n. \tag{8}$$

A challenge is derived by Fiat-Shamir [8] over the entire statement, before any opening:

$$\mathcal{O} = \mathsf{select}_o\Big(H\big(\, a \,\|\, hS \,\|\, C_0 \cdots C_{k-1} \,\|\, E_1 \cdots E_n \,\|\, \mathsf{params}\,\big)\Big). \tag{9}$$

For each $i \in \mathcal{O}$ the prover reveals $f(i)$ and $\rho_i$, and the verifier performs two checks:

$$[f(i)]P_1 \stackrel{?}{=} \textstyle\sum_{j=0}^{k-1} [\, i^j \,] C_j \tag{10}$$

$$E_i \stackrel{?}{=} \mathsf{Enc}_{\mathsf{id}_a}(f(i);\, \rho_i) \tag{11}$$

Equation 10 binds the share to the polynomial. Equation 11 binds the polynomial to the published bytes. Omitting the second is the most likely implementation error, because the first looks sufficient and is not. The verifier also checks $C_0 = hS$, which is the join back to $\mathcal{R}$. Figure 3 shows one round.

### 3.7 Custody

The escrow master secret is held by a public threshold network that the platform does not operate and does not pay per transaction. Several such networks run today, offering release conditioned on observable on-chain events. The platform holds no share of $\mathsf{msk}$.

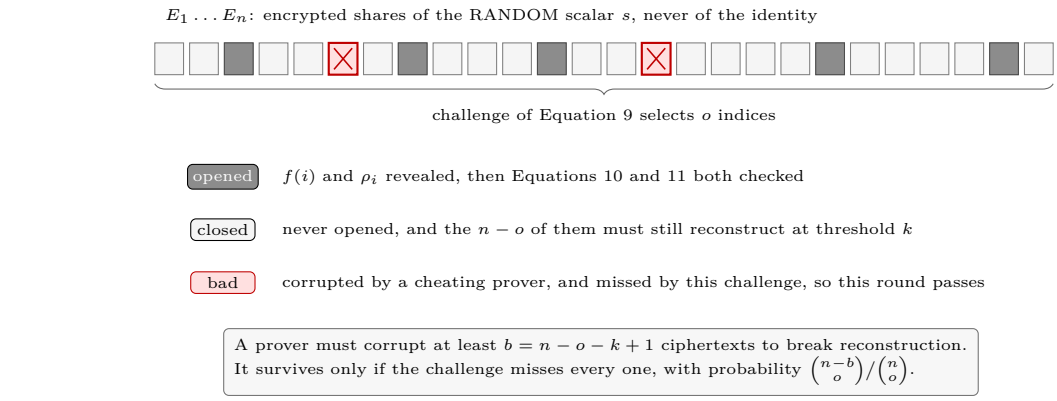


Figure 3: One round of the cut-and-choose, drawn with 26 cells for legibility. The reference profile uses $n = 130$, $k = 27$ and $o = 26$. Opening $o = 26$ reveals fewer shares than the threshold $k = 27$, so the openings leak nothing about $s$.

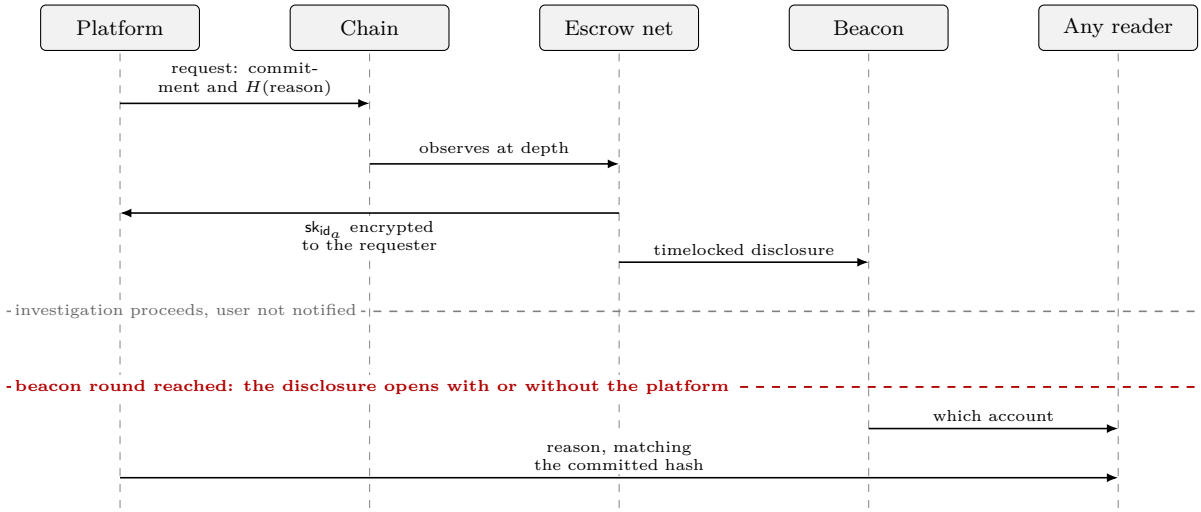


Figure 4: Unsealing. The count of requests is public from the first message. The target becomes public when the beacon reaches the sealed round, whether or not the platform cooperates. The reason was fixed by its hash at request time, so it cannot be revised once the outcome is known.

A deployment may spread shares across several independent networks and require all of them, which raises the bar for collusion at the cost of survivability if one ceases operation. Section 9.1 argues this is the most consequential deployment parameter.

### 3.8 Unsealing and disclosure

Figure 4 gives the sequence and its timing.

The escrow network encrypts $\mathsf{sk}_{\mathsf{id}_a}$ to the requester key committed in the same transaction, never publishing it in the clear, because a published key would expose the user's identity to everyone rather than to the authority that asked.

Recovery is not simply decrypt and reconstruct. Given $\mathsf{sk}_{\mathsf{id}_a}$, a recovering party computes $\sigma_i = \mathsf{Dec}_{\mathsf{sk}}(E_i)$ for successive $i \notin \mathcal{O}$, discards any $\sigma_i = \perp$, retains only those satisfying Equation 10, and stops once it holds $k$ of them, indexed by a set $S$. It then interpolates

$$s' = \sum_{i \in S} \sigma_i \prod_{j \in S \setminus \{i\}} \frac{j}{j - i} \pmod{r}, \tag{12}$$

and accepts only if $[s']P_1 = hS$. Taking the first $k$ decryptions without checking each against Equation 10 is unsafe, because cut-and-choose does not promise that every unopened ciphertext is well formed. At most $n - o + 1$ decryptions are needed.

### 3.9 Client integrity

The application holds the plaintext while it works, so no cryptography prevents it from transmitting it. What the design provides is detectability. Platform attestation gates enrollment, so a repackaged application cannot enroll, which defends against third-party tampering and provides nothing against a hostile first party.

Three checks are available to a user without expertise. Observe the application's network traffic, which is why certificates are not pinned. Enroll with networking disabled, since sealing requires none. Rebuild from published source where the platform permits it.

## 4 Security analysis

### 4.1 Soundness of the escrow proof

A cheating prover corrupts some set of $b$ ciphertexts, hoping the challenge misses all of them. Reconstruction from the unopened set requires $k$ good shares out of $n - o$, so breaking recovery requires

$$b \;\geq\; n - o - k + 1. \tag{13}$$

Such a prover passes only when $\mathcal{O}$ avoids every corrupted index, which happens with probability

$$\Pr[\text{pass}] \;=\; \binom{n-b}{o} \Big/ \binom{n}{o}, \tag{14}$$

giving soundness in bits

$$\lambda \;=\; \log_2 \binom{n}{o} \;-\; \log_2 \binom{n-b}{o}. \tag{15}$$

At the reference profile $n = 130$, $k = 27$, $o = 26$, Equation 13 gives $b = 78$ and Equation 15 gives $\lambda = 41.51$ bits.

Figure 5 plots Equation 14 against $b$ and shows why the scheme is not weak at small $b$ in any way that matters. Corrupting a single ciphertext succeeds with probability 0.80, and achieves nothing, because 129 good ciphertexts remain and recovery still succeeds. Only at $b = 78$ does success mean the secret is unrecoverable, and there the probability is $3.19 \times 10^{-13}$.

Equation 15 gives pre-query soundness. A prover may discard a transcript and retry with fresh randomness until it draws a favourable challenge, so a deployment must include grinding in its budget: with $q$ attempts the bound degrades to roughly $q \cdot 2^{-\lambda}$. At the reference profile each attempt requires recomputing $n$ pairing-based encryptions, which makes a useful $q$ economically infeasible, but the figure must be labelled for what it is.

Table 2 and Figure 6 give the parameter space.

### 4.2 Measured cost of the escrow proof

Table 1 reports the reference implementation at the reference profile. These are measurements rather than estimates, and unlike the soundness figures they are host specific.

Two observations. Recovery cost is dominated by pairing operations and scales with $k$ rather than $n$, which is why it used 27 decryptions and not 105. And the measured record size of about

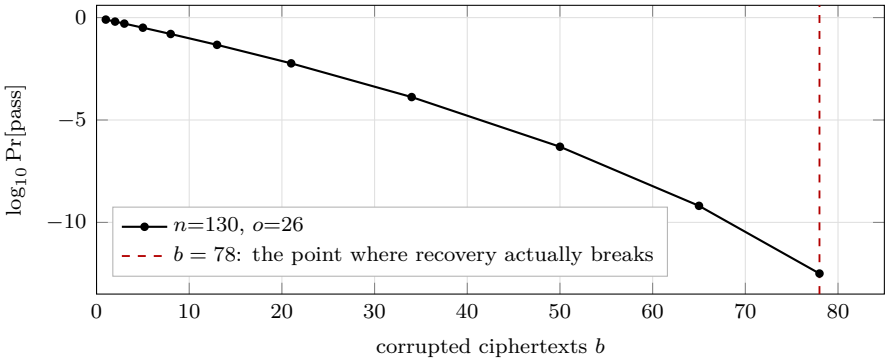


Figure 5: Probability that a cheating prover survives one round, against how many ciphertexts it corrupted. Low $b$ passes easily and accomplishes nothing. Recovery only fails from $b = 78$, where the probability is $3.19 \times 10^{-13}$.

| Operation, $n$=130, $k$=27, $o$=26 | Cost |
|---|---|
| Prove, on the user's device | 1754 ms |
| Verify, on the platform | 2335 ms |
| Recover, given the derived key | 2309 ms |
| Decryptions used during recovery | 27 of at most 105 |
| Enrollment record size | ≈ 30,500 bytes |
| Seal on device, whole enrollment | 1716 ms |
| Platform accept, both proofs | 2330 ms |

Table 1: Measured on an Apple M5 Pro, Node v24.18.0. Recovery needed 27 decryptions against a worst case of 105, because it stops as soon as it holds $k$ shares that pass Equation 10. The record is an order of magnitude smaller than the 50 KB the cost model of Section 8 assumes, so that model is conservative.

30 KB is well under the 50 KB the cost model assumes, so the per-registration figures in Section 8 overstate storage by roughly forty percent.

| Profile | $b$ | Soundness (bits) | Worst-case decryptions |
|---|---|---|---|
| $n$=24, $k$=7, $o$=6 | 12 | 7.19 | 19 |
| $n$=120, $k$=25, $o$=24 | 72 | 38.29 | 97 |
| $n$=126, $k$=26, $o$=25 | 76 | 40.22 | 102 |
| $n$=130, $k$=27, $o$=26 | 78 | 41.51 | 105 |

Table 2: Escrow proof parameters, from Equations 13 and 15. The last row is the reference profile.

### 4.3 Confidentiality before release

**Proposition 2** (The openings leak nothing)**.** *The $o$ revealed shares are values of a degree $k-1$ polynomial at $o < k$ distinct points. For any candidate $s^* \in \mathbb{Z}_r$ there is exactly one polynomial of degree $k-1$ agreeing with those shares and having $f(0) = s^*$. The openings are therefore independent of $s$.*

The Feldman commitments do reveal $C_0 = [s]P_1$, so $s$ is only as hidden as discrete log in $\mathbb{G}_1$. This is exactly why $s$ must be a uniform scalar and not the identity. A commitment to a

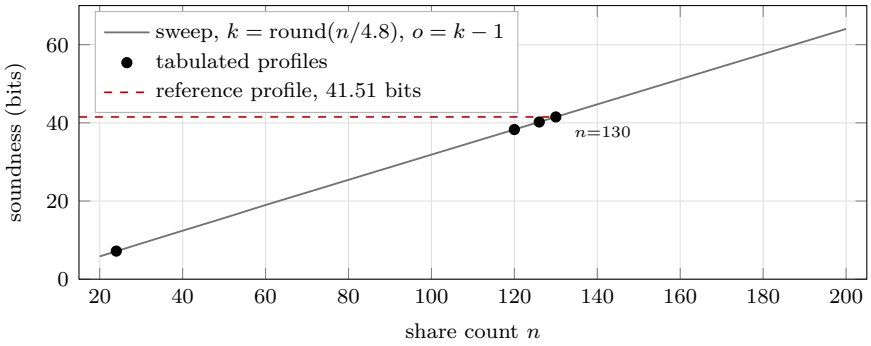


Figure 6: Soundness against share count, from Equation 15. Linear in $n$ once the opened fraction is fixed. One more bit costs about 3.1 more shares and about 2.5 more decryptions at recovery.

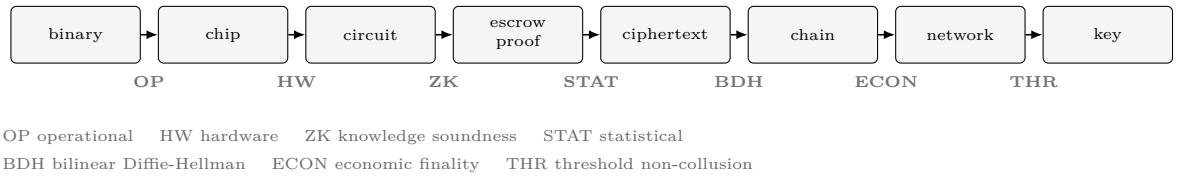


Figure 7: The chain of custody. Each arrow carries the kind of assumption that protects it. Only three of the seven are cryptographic in the strict sense, and the construction is only as strong as the weakest.

low-entropy value is a target for offline enumeration, and identity data is low entropy: document numbers are short, and names and birth dates already appear in breach corpora. Sealing the identity under $\mathsf{kdf}(s)$ and committing only to $s$ removes that attack.

**Proposition 3** (Platform confidentiality)**.** *Under A1 and A3, a platform holding* $\{E_i\}$, $\{C_j\}$, *hS and* $\mathsf{ct}$ *learns nothing about the identity beyond the length of* $\mathsf{ct}$*.*

Recovering the identity requires $\mathsf{kdf}(s)$, hence $s$, hence $k$ shares, hence $\mathsf{sk}_{\mathsf{id}_a}$, hence a threshold of $\mathsf{msk}$ shares under Equation 1. Deriving $\mathsf{sk}_{\mathsf{id}_a}$ from $\mathsf{mpk}$ alone is the bilinear Diffie-Hellman problem.

### 4.4 Per-account scoping

**Proposition 4** (A key opens one account)**.** $\mathsf{sk}_{\mathsf{id}_a}$ *decrypts ciphertexts under* $\mathsf{id}_a$ *and no others. Using it against* $\mathsf{id}_{a'}$ *for* $a' \neq a$ *requires* $[\mathsf{msk}]H_1(\mathsf{id}_{a'})$*, which by Equation 1 is a different group element with no efficient relation to the first.*

This is the property P2 depends on. Opening $N$ accounts requires $N$ derivations, each triggered by its own public request. Bulk access produces bulk public evidence, and no key exists whose possession opens more than one person.

### 4.5 The chain of custody, link by link

Figure 7 follows one enrollment from the application binary to a released key, and Table 3 states what secures each step.

### 4.6 What is not cryptographic

Four assumptions carry weight and none of them is mathematics.

| Link | What it protects | Mechanism | Kind |
|---|---|---|---|
| Binary to device | the code that runs is the code published | reproducible builds, store signature, attestation | operational |
| Chip to app | the passport is present, not copied | Equations 3, 4: chip signs a fresh bound challenge | hardware |
| App to circuit | the sealed bytes are the state-signed identity | Equations 2, 7 share witness wires | cryptographic |
| Circuit to escrow | the committed secret is recoverable | Equations 10, 11, and $C_0 = hS$ | statistical |
| Escrow to storage | the platform cannot read what it holds | Equation 1, bilinear Diffie-Hellman | cryptographic |
| Request to chain | the request cannot be hidden or reordered | chain finality at depth | economic |
| Chain to network | no key exists without a public request | threshold derivation on an observed event | threshold |
| Key to disclosure | the opening becomes public | timelock beacon, threshold signature | threshold |
| Record to history | published history cannot be rewritten | Merkle inclusion plus anchoring | crypto, econ |

Table 3: What secures each link. A reader tracing one enrollment passes through all nine.

A1, threshold non-collusion, is what the construction rests on. It replaces the custodial assumption of Section 2, and it is better only because a user can inspect who operates the nodes and under which jurisdictions, which they cannot do for a platform's internal key management.

A2, chain finality, is economic. It holds because reversing a settled block costs more than an attacker gains, not because it is impossible.

A5, chip key non-extractability, is a hardware claim. Chips offering only passive authentication are clonable from a data dump, which is why Equation 4 is required rather than optional. Relay attacks remain possible even with it.

A6, client integrity, cannot be discharged by cryptography at all. The application legitimately handles the plaintext. Section 3.9 makes violations detectable and never impossible.

# 5 Design decisions

Each of the following prevents a specific failure.

1. **The shared secret is a uniform scalar, never the identity.** A Feldman commitment reveals $[s]P_1$, so committing to low-entropy data invites offline enumeration.

2. **The verifier checks $C_0 = hS$ explicitly.** Making the commitment a circuit output stops the prover choosing it freely, and does not by itself bind it to the commitment the escrow proof uses.

3. **Opened shares require Equation 11, not only Equation 10.** Checking a share against its commitment says nothing about the bytes stored beside it.

4. **Recovery validates every candidate**, per Equation 12 and the check that follows it.

5. **The challenge covers the complete statement before any opening**, per Equation 9, so openings never influence selection.

6. **A fresh polynomial per enrollment attempt.** Otherwise opened subsets from repeated attempts accumulate and cross the threshold.

7. **The nullifier excludes the account identifier**, per Equation 5. Including it would let one document enroll unlimited accounts.

8. **Nobody validates the legal merit of a request.** No contract and no threshold network can read a court order. A fabricated justification is a credibility failure rather than a cryptographic one, detectable once the disclosure opens.

One input remains unbound by construction. A liveness check comparing a selfie against the chip photograph produces a single bit, and a circuit proving that bit was set proves nothing about how it was produced. It is a client-trusted input and must be labelled as one.

# 6 The settlement layer

The construction needs a chain for two jobs: anchoring the transparency record, and carrying unseal requests where the escrow network can observe them. The second constrains the first, because the request must land somewhere the network already watches.

We take Gnosis Chain as the reference deployment. It is a proof-of-stake EVM chain secured by more than 140,000 validators, a set second in size only to Ethereum, reached by setting the stake per validator low enough for broad participation. Blocks arrive about every five seconds, and the network tracks Ethereum's upgrade path.

Two properties matter here beyond the usual ones.

**Costs are flat in fiat.** Gas is paid in a stablecoin bridged one to one, so an anchor costs the same next year as this year. A chain whose gas is denominated in a volatile asset turns a ten-year anchoring commitment into an unhedged position on that asset's price. For infrastructure meant to outlast several market cycles, predictable beats cheap, and here it is both.

**The escrow network already observes it.** Conditional key release requires the network to evaluate a condition against chain state. Networks capable of this watch specific chains, and a chain outside that set would need an oracle to bridge it, reintroducing the trusted intermediary the design exists to remove.

**The honest trade.** Gnosis carries less economic weight behind finality than Ethereum itself. For anchoring a transparency record, where the property required is that published history cannot be quietly rewritten, that margin is adequate. A deployment wanting stronger permanence can anchor continuously on the settlement chain and push a periodic root to a higher-security chain through a free timestamping aggregator, paying for the stronger guarantee only at the slower cadence.

# 7 Cost

The commercial argument is that document verification happens on hardware the user already owns. Constants: 50,000 gas for an anchor write, 120,000 for an unseal request, gas at 2 gwei, 50 KB per enrollment record, storage at `EUR` 0.021 per GB-month, and 0.3 seconds of processing per registration. Deployments anchor on a size trigger or a time trigger, whichever fires first, so the model takes the maximum of the two. Figure 8 and Table 4 give the result.

Total annual chain spend never exceeds `EUR` 9.20 across this grid, the worst case being ten million registrations with hourly anchoring. Daily anchoring at the same volume costs `EUR` 0.92 per year.

Chain cost is therefore not a design constraint at this scale, and anchor cadence should be chosen for latency rather than price. Past roughly one hundred thousand registrations a

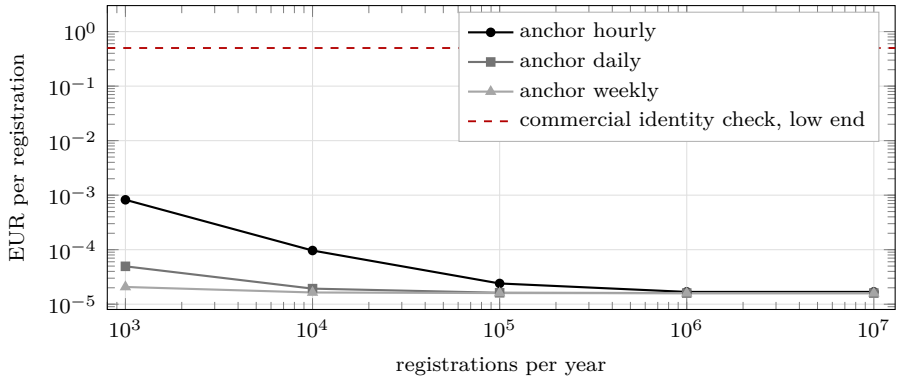


Figure 8: Cost per registration against volume, log-log. Curves flatten once the anchor cost amortises, after which storage dominates. The gap to commercial identity verification is four to five orders of magnitude.

| Anchor policy | 1k | 10k | 100k | 1M | 10M |
|---|---|---|---|---|---|
| Hourly | $8.22 \times 10^{-4}$ | $9.65 \times 10^{-5}$ | $2.40 \times 10^{-5}$ | $1.68 \times 10^{-5}$ | $1.68 \times 10^{-5}$ |
| Daily | $4.95 \times 10^{-5}$ | $1.93 \times 10^{-5}$ | $1.62 \times 10^{-5}$ | $1.60 \times 10^{-5}$ | $1.60 \times 10^{-5}$ |
| Weekly | $2.07 \times 10^{-5}$ | $1.64 \times 10^{-5}$ | $1.60 \times 10^{-5}$ | $1.59 \times 10^{-5}$ | $1.59 \times 10^{-5}$ |

Table 4: EUR per registration by anchor cadence and annual volume.

year, storage dominates, at which point the total for ten million registrations is around `EUR` 160 annually.

The weakest constant is the 50 KB record size, derived from the structure of pairing-based ciphertexts rather than measured. A four-fold error triples the total and leaves the conclusion unchanged.

## 8 The disclosure reference

Property P3 asks that the published record carry a reference a human can check. That reference is a free-form string, deliberately.

**Why a string and not a structure.** The reference exists to be read by a person holding a document and asking whether the two match. A rigid schema would have to anticipate the request formats of every jurisdiction a platform operates in, and would break the moment one changed. What the design fixes is not the format but the binding: the hash of the string is committed on chain at request time, so whatever it says, it was said before the outcome was known.

**One convention worth requiring.** Requests reaching a platform fall into two broad categories in most jurisdictions. Some are authorised by a judge and carry a case identifier issued by a court registry, stable across the proceeding. Others are issued directly by prosecutors or police under statutory powers, carry only an internal administrative reference, and involve no judicial authorisation at all. These differ enormously in the accountability they represent.

A leading token naming the category is therefore worth requiring, even though nothing parses it:

```
{ request: 'judicial <registry case identifier>' }
{ request: 'administrative <issuing body, internal reference>' }
```

Published counts should separate the two. A single total that averages a judge's order together with an administrative form tells a reader something untrue, and the ratio between them is the number that actually describes a platform's exposure.

**What the reference does not prove.** A string is checkable only against a registry the reader can query, and investigation records are frequently sealed. Two things partly compensate. The affected user acquires standing once their account is suspended and can ask about their own case, which is the check by the party most motivated to make it. And where an authority publishes aggregate counts of orders issued, a platform's published total exceeding that figure is proof of misstatement without needing any individual record.

**The tension to resolve before deployment.** Statutory duties to comply are sometimes paired with duties not to disclose. An architecture that publishes by construction can collide with that, and the resolution determines whether the on-chain commitment can be immediate or whether the whole request must sit behind the timelock. That is a question for counsel in each jurisdiction, and it is the first question a deployment should ask.

# 9 Discussion

## 9.1 What CLAVE does not provide

The implementation covers enrollment, the escrow proof, key derivation, recovery, the unseal request and the timelocked disclosure. It does not cover three things. The credential circuit is evaluated in the clear, so no zero-knowledge property holds and the binding of Equations 2 to 7 is what the tests exercise. The settlement layer runs against a mock that refuses to write to a real chain, because a deployment has to name a contract first. The escrow network is a single process rather than a threshold of independent nodes, which models the interface and not the custody.

Soundness figures are computed from Equation 15 and are exact. Cost figures come from the model of Section 8. Timings are host specific.

The escrow network can collude. A threshold of its nodes can derive keys without a public request, and no observer would see anything. This is A1.

Networks capable of conditional key release are young. Asking one to hold identity escrow for a decade is the principal deployment risk, larger in our assessment than any cryptographic concern in this paper. Spreading shares across several independent networks is the only lever that reduces it without recruiting anyone, and it trades collusion resistance against the risk that one network ceasing operation renders records permanently unopenable.

The credential circuit is the critical path and does not exist. Proving Equations 2 through 7 together requires verifying a national signature scheme and an authenticated encryption inside a circuit, which is feasible and not cheap.

The client remains in the trusted base. Reproducible builds, transparency and audit make tampering detectable, never impossible.

Finally, CLAVE detects rather than prevents. A monitor that proves a platform lied has produced a document. Whether that document carries a consequence is a question of law.

## 9.2 Related work

Certificate Transparency [2] and CONIKS [3] establish detection over prevention and supply the log machinery. Transparency overlays [4] generalise it. Verifiable encryption of discrete logarithms [9] addressed escrow with sigma protocols two decades ago and remains the closest prior treatment of the enrollment problem. Identity-based encryption [5] supplies Equation 1.

Threshold timelock encryption [10] supplies the disclosure mechanism. Shamir sharing [6] and Feldman commitments [7] supply the escrow proof, and Fiat-Shamir [8] makes it non-interactive.

### 9.3 Conclusion

A platform that holds your identity cannot prove it did not read it. That is not an engineering shortfall and no amount of logging repairs it.

CLAVE follows from taking that seriously. Seal on the user's device. Hold no master key. Make every release depend on a public act the platform cannot perform privately. What remains is an availability question, and availability is something anyone can observe.

The cryptography is the straightforward part. Deciding what not to hold is the rest of it.

## Acknowledgments

This design was refined against adversarial review that repeatedly contradicted its author. Several decisions in Section 6 exist because an earlier draft was wrong in a way that would have been expensive to discover later.

## Author contributions

R.S. designed the construction and set the evidence standard. Design review and drafting used large language model assistance under human direction, with independent models used to review each other's conclusions. The author is responsible for all content.

## Availability

The reference implementation, the cost model, and the reproduction harness are at `https://clave.sospedra.me` and in the repository at `https://github.com/sospedra/sospedra.me`. Running `scripts/reproduce.mts` emits Table 2, Figure 5 and Table 1. Running `scripts/cost-model.mts` emits Table 4 and Figure 8. Running `pnpm test` exercises the protocol end to end against a mock chain and a mock beacon.

## References

[1] H. Abelson *et al.*, "Keys under doormats: mandating insecurity by requiring government access to all data and communications," *Journal of Cybersecurity*, vol. 1, no. 1, pp. 69–79, 2015.

[2] B. Laurie, E. Messeri, and R. Stradling, "Certificate Transparency version 2.0," RFC 9162, Dec. 2021.

[3] M. S. Melara, A. Blankstein, J. Bonneau, E. W. Felten, and M. J. Freedman, "CONIKS: bringing key transparency to end users," in *Proc. USENIX Security*, 2015, pp. 383–398.

[4] M. Chase and S. Meiklejohn, "Transparency overlays and applications," in *Proc. ACM CCS*, 2016, pp. 168–179.

[5] D. Boneh and M. Franklin, "Identity-based encryption from the Weil pairing," in *Proc. CRYPTO*, 2001, pp. 213–229.

[6] A. Shamir, "How to share a secret," *Communications of the ACM*, vol. 22, no. 11, pp. 612–613, 1979.

[7] P. Feldman, "A practical scheme for non-interactive verifiable secret sharing," in *Proc. IEEE FOCS*, 1987, pp. 427–438.

[8] A. Fiat and A. Shamir, "How to prove yourself: practical solutions to identification and signature problems," in *Proc. CRYPTO*, 1986, pp. 186–194.

[9] J. Camenisch and V. Shoup, "Practical verifiable encryption and decryption of discrete logarithms," in *Proc. CRYPTO*, 2003, pp. 126–144.

[10] N. Gailly, K. Melissaris, and Y. Romailler, "tlock: practical timelock encryption from threshold BLS," IACR ePrint 2023/189, 2023.